Phase 1 — Unity Factory Scene: Detailed Build Plan

This phase produces a working factory floor driven by your existing DES, with no ML connectivity yet. By the end you can load a Taillard instance, pick a dispatching rule, hit Play, and watch jobs flow through machines with the correct makespan.

Step 1: Scene Skeleton and Grid Layout

Goal: An empty factory floor with machines placed on a spatial grid.

1. Create a new Unity scene called `FactoryFloor`.
2. Build a flat ground plane sized to your grid. Since your distance matrix is 8×8, start with 8 machines. Use a world-space scale where 1 unit = 1 grid cell (e.g. a 20×20 unit floor gives room for 8 machines with aisles between them).
3. Create a `MachinePrefab` — a simple 3D object (cube or low-poly model) with:
   - A `MachineVisual.cs` MonoBehaviour holding a reference to the core `Machine` from your DES.
   - A `MeshRenderer` whose material color will reflect state.
   - A world-space UI canvas above it showing machine ID and current status text.
4. Write a `FactoryLayoutManager.cs` that:
   - Reads the machine count from the loaded `TaillardInstance`.
   - Instantiates `MachinePrefab` instances at predefined grid positions (hardcode a layout table for 8, 15, and 20 machines — you can refine later).
   - Populates the 8×8 pairwise distance matrix from actual world-space positions (`Vector3.Distance` between transforms), then flattens it to 64-D. Store this — it feeds directly into your observation space later.
5. Add a simple free-look camera (Cinemachine or a basic orbit script) so you can watch the floor during playback.

Validation: Hit Play. You see 8 machine cubes on a grid, each labelled, all showing "IDLE". The distance matrix logs to console with correct values.

Step 2: Machine State Machine (Visual Layer)

Goal: Machines visually reflect their DES state transitions.

1. Define a `MachineState` enum: `Idle`, `Busy`, `Failed`, `Repairing`. This mirrors what your `Machine.cs` already tracks internally.
2. In `MachineVisual.cs`, implement a `SetState(MachineState state)` method that:
   - Changes the material color (e.g. green=idle, yellow=busy, red=failed, blue=repairing).
   - Updates the overhead status text.
   - Optionally triggers a small animation (pulse on state change, particle burst on failure).
3. Add a progress bar (world-space UI slider) above each machine showing operation completion percentage. Drive it from `(currentTime - opStartTime) / opDuration`.
4. Don't wire this to the DES yet — just test it manually. Add a debug button or inspector toggle that cycles through states on click.

Validation: Click a machine in the editor, cycle its state, see color and text update instantly. Progress bar fills smoothly when set to Busy.

Step 3: Job and Operation Data Flow

Goal: Jobs exist as GameObjects that carry their operation sequence from the DES.

1. Create a `JobPrefab` — a small colored sphere or capsule. Each job gets a unique color (use a palette or hash the job ID to a hue).
2. Write a `JobVisual.cs` MonoBehaviour that holds:
   - A reference to the core `Job` object from your DES.
   - The current operation index.
   - Visual state: `Queued`, `InTransit`, `Processing`, `Complete`.
3. Write a `JobManager.cs` (scene singleton) that:
   - On episode start, reads all jobs from the loaded `TaillardInstance` and instantiates `JobPrefab` instances.
   - Parks all jobs at a "spawn area" off to the side of the factory floor.
   - Exposes a method `OnJobDispatched(int jobId, int machineId)` that the DES bridge will call — for now, just log it.
4. Jobs don't move yet. This step just ensures every DES job has a visual counterpart with the right metadata.

Validation: Load a ta01 instance (15 jobs × 15 machines). Hit Play. 15 colored spheres appear in the spawn area. Console logs confirm each job's operation sequence matches the Taillard data.

Step 4: NavMesh and AGV Pathfinding

Goal: AGVs can navigate between any two machines on the factory floor.

1. Bake a NavMesh on your ground plane. Mark machine prefabs as NavMesh obstacles so AGVs route around them.
2. Create an `AGVPrefab` — a small flat disc or cart model with a `NavMeshAgent` component.
3. Write an `AGVController.cs` that:
   - Accepts a `MoveTo(Vector3 target, System.Action onArrival)` command.
   - Uses `NavMeshAgent.SetDestination` to navigate.
   - Calls the `onArrival` callback when `remainingDistance < threshold`.
   - Exposes its current position for grid rasterization later (Phase 2).
4. Write an `AGVPool.cs` that manages a small fleet (start with 2–4 AGVs). It assigns the nearest idle AGV to a transport request and queues requests if all AGVs are busy.
5. Test manually: in the inspector, click "Send AGV from Machine 0 to Machine 5" and watch it pathfind around obstacles.

Validation: AGVs navigate smoothly between machines, avoid obstacles, and the arrival callback fires correctly. Multiple queued requests are serviced in order.

Step 5: DES-to-Unity Bridge (The Core Integration)

Goal: The DES simulator drives all visual activity. Unity renders; DES decides.

This is the most critical step. Your `DESSimulator.cs` uses an `EventQueue` that processes events in timestamp order. You need to replay this in Unity time.

1. Write a `SimulationBridge.cs` MonoBehaviour that:
   - Holds a reference to a `DESSimulator` instance.
   - On `Start()`, loads the selected `TaillardInstance` and initialises the DES with a chosen dispatching rule.
   - Does **not** run the DES to completion upfront. Instead, processes events incrementally.
2. Implement a **stepped execution model**. There are two clean approaches — pick whichever fits your DES architecture better: **Option A — Event-at-a-time with visual delay:**

```
Each frame (or on a timer):
  1. Peek at the next DES event timestamp.
  2. Advance Unity's sim clock to that timestamp.
  3. Pop and process the event in the DES.
  4. Translate the event into a Unity visual command.
  5. Wait for the visual to complete (AGV arrives, progress bar fills)
     before processing the next event.
```

This gives a real-time animated playback but runs slower than wall clock. Good for visual debugging.

Option B — Batch-then-replay:

```
On Start():
  1. Run the DES to completion (instant).
  2. Record every event as a timestamped visual command
     (job X dispatched to machine Y at time T).
  3. Replay the command log in Unity time, using coroutines
     or a timeline to trigger visuals at the right sim-time.
```

This is simpler to implement and guarantees the schedule is correct before you try to visualise it. Recommended for Phase 1.

3. Define a `VisualCommand` struct that captures every event type your DES produces:
   - `JobDispatched(jobId, machineId, startTime)`
   - `OperationStarted(jobId, machineId, startTime, duration)`
   - `OperationCompleted(jobId, machineId, endTime)`
   - `MachineFailure(machineId, time)` (if you add breakdowns)
   - `MachineRepaired(machineId, time)`
4. Write the replay coroutine that walks through the command list:

```
Sort commands by timestamp.
For each command:
  - Wait until Unity sim clock >= command.time
    (use a configurable speed multiplier so you can run at 1×, 5×, 20×).
  - Execute the visual action:
      JobDispatched → Tell AGVPool to transport job from current
                      location to target machine.
      OperationStarted → Machine.SetState(Busy), job snaps to
                         machine position, progress bar begins.
      OperationCompleted → Machine.SetState(Idle), job moves to
                           next queue or "complete" area.
```

5. Add a speed slider in the UI: 1× (real-time relative to sim clock), 10×, and "instant" (skip all waits, just teleport jobs).

Validation: Load ta01, select SPT rule, hit Play. Jobs visually move between machines in the correct order. The final sim-time displayed in the UI matches the makespan your `TaillardValidator` reports for the same instance and rule. If they don't match, the bridge has a bug — the DES output is your ground truth.

Step 6: Queue Visualisation

Goal: See jobs waiting at each machine, not just the one being processed.

1. At each machine, define a "queue area" — a short line of positions extending from the machine (like a conveyor belt or a row of parking spots).
2. When a job arrives at a machine that's already busy, place the job's visual at the next open queue slot.
3. When the machine finishes and picks the next job (according to the dispatching rule), move that job from the queue into the "processing" position on the machine.
4. Update the queue visually whenever a job enters or leaves — shift remaining jobs forward.
5. Display the queue length as a number on the machine's overhead UI.

Validation: On a busy instance, you should see queues building up at bottleneck machines and draining at underutilised ones. Queue lengths should match the `avg_queue_len` you'll compute as a global scalar later.

Step 7: Job Lifecycle and Completion

Goal: Jobs visually complete their full lifecycle from spawn to finish.

1. Track each job's progress through its operation list. After the final operation completes, move the job to a "completed jobs" area (a corner of the factory floor, or fade it out).
2. Maintain a running `JobCompletionRate` counter in `JobManager`: completed jobs / total jobs. Display it on a simple HUD.
3. Colour-code job urgency or remaining work if you want richer visuals — e.g., jobs with more remaining operations are more saturated.
4. At episode end (all jobs complete), display the final makespan in the HUD and compare it to the known optimal from the Taillard metadata.

Validation: All jobs complete. The makespan displayed equals the DES-computed makespan. No job is left stuck in a queue or transit.

Step 8: Event-Driven Decision Points (Prep for Phase 3)

Goal: Identify and mark the exact moments where an ML agent would make a decision, even though Phase 1 uses fixed dispatching rules.

1. In your `SimulationBridge`, flag every `OperationCompleted` event where the machine has a non-empty queue (meaning a dispatching decision must be made about which queued job to process next).
2. Log these decision points: `[t=245] Machine 3 free, queue=[Job 7, Job 12, Job 3], rule chose Job 12 (SPT)`.
3. Add a visual indicator — briefly flash the machine or pause the replay at decision points (togglable). This helps you visually verify that decisions happen at the right times and frequencies.
4. Count total decision points per episode. This tells you how many actions the RL agent will take per episode, which matters for rollout buffer sizing.

Validation: The number and timing of decision points matches what you'd expect from the problem structure. A 15×15 instance should have roughly 200–225 decision points (each of the 225 operations triggers a potential dispatch).

Step 9: UI and Debug Tooling

Goal: An inspector/HUD that makes Phase 1 actually useful for debugging the later phases.

1. **HUD elements:**
    - Current sim time and speed multiplier.
    - Makespan (running and final).
    - Active / waiting / completed job counts.
    - Active dispatching rule name.
2. **Inspector panel** (on `SimulationBridge`):
    - Dropdown for Taillard instance selection (ta01–ta80).
    - Dropdown for dispatching rule (all 8 composite PDRs plus any singles).
    - "Run" button that resets and starts a new episode.
    - "Step" button that advances one event at a time.
3. **Clickable machines:** Click a machine to see its full history (what it processed, when, queue stats) in a side panel or console log.
4. **Gantt chart overlay (optional but valuable):** Render a simple runtime Gantt chart using Unity UI. Each row is a machine, coloured blocks are operations. This directly mirrors what scheduling researchers look at and makes correctness obvious at a glance.

Validation: You can interactively explore any instance + rule combination, step through events, and visually confirm correctness without touching code.

Step 10: Multi-Instance Validation Sweep

Goal: Confidence that the visual simulation matches the DES across many instances.

1. Use your existing `CrossValidator.cs` to batch-run all Taillard instances with all dispatching rules, collecting makespans.
2. Run the same instances through the visual `SimulationBridge` (can be done in "instant" speed mode) and collect makespans.
3. Assert that every makespan matches exactly. Any discrepancy means the bridge introduced a bug (event ordering, timing rounding, queue handling).
4. Cross-check against your Python reference (`reference_makespans.csv`) to triple-validate.

Validation: 100% match across all instances and rules between DES-only, visual bridge, and Python reference. Phase 1 is complete.

Suggested Time Estimates

Step	Description	Estimate
1	Scene skeleton and grid layout	1–2 days
2	Machine state machine visuals	0.5–1 day
3	Job and operation data flow	1 day
4	NavMesh and AGV pathfinding	1–2 days
5	DES-to-Unity bridge	2–3 days
6	Queue visualisation	0.5–1 day
7	Job lifecycle and completion	0.5–1 day
8	Decision point marking	0.5 day
9	UI and debug tooling	1–2 days
10	Multi-instance validation sweep	1 day
Total		**~9–14 days**

Step 5 (the DES bridge) is the load-bearing piece. If you prototype it early — even with placeholder visuals — everything else snaps into place around it. Steps 1–4 can be built in parallel by setting up the visual prefabs while you design the bridge architecture on paper.